

Snort RAG Rule Generator

Final Codex Fix and Evidence Handoff

Purpose: finish the remaining code-side polish, regenerate clean artifacts, and prepare the project for the manual runtime steps: real LLM benchmarking and real Snort/PCAP validation.

Field	Value
Project	snort_rag_rule_generator
Target	Move current strong implementation from approximately 91-93/100 toward near-final submission quality
Main remaining risk	Evidence gaps: real LLM execution and real Snort runtime/PCAP execution
Core rule	Do not fake runtime success. SKIPPED is acceptable until the user runs Snort/Ollama locally.
Audience	Codex implementation agent + project owner after Codex finishes

Important honesty statement: Codex must not make the project look artificially perfect. The priority is auditable evidence: generated prompts, model names, retrieved source IDs, validation results, Snort command outputs, PCAP alert counts, and clear limitations.

1. Executive Summary

The project is now much stronger than the old 78/100 version. The controlled RAG + LLM architecture, clustering module, benchmark scripts, PCAP generation, Snort validation scripts, dashboard transparency, and compliance documentation are largely in place. The remaining work is not mainly architectural; it is about cleaning artifacts, strengthening consistency, and preparing manual execution evidence.

Area	Current status	Required final action
Controlled RAG + LLM	Implemented with mock benchmark and fallback	Regenerate artifacts and make real model execution easy
Real LLM benchmark	Mock benchmark only	User must run Ollama/local models after Codex polish
Snort runtime validation	Scripts exist but CSV says SKIPPED	Codex improves scripts/docs; user runs with real Snort
PCAP/FP evidence	Synthetic PCAP files generated; replay skipped	Codex improves reporting; user runs replay with Snort
Generated artifacts	Some old CSV rows may still show duplicate source IDs or old explanation mismatch	Codex regenerates and verifies clean artifacts
Notebook/report wording	Some old wording may still say APIs/Ollama forbidden	Codex updates wording to controlled RAG-only LLM usage

Expected result after Codex: the repository should be code-clean and evidence-ready. If Snort/Ollama are not available in Codex, the outputs must remain honestly marked as SKIPPED or mock-only, with precise commands for the user to run locally.

2. Priority Matrix

Priority	Category	Owner	Goal
P0	Clean stale artifacts and consistency errors	Codex	Regenerate results and remove duplicate source IDs, classtype mismatch, old wording
P0	Runtime script robustness	Codex	Make Snort scripts produce clear PASS/FAIL/SKIPPED metadata and commands
P0	Real LLM command readiness	Codex	Make Ollama model benchmarking easy and documented
P0	Final report limitation wording	Codex	No claim of real Snort/LLM success unless executed
P1	Dashboard/report polish	Codex	Expose model, prompt, raw output, retrieved docs, validation, FP metadata
P1	Submission package checklist	Codex	Create optional packaging script and final checklist
Manual	Run local Ollama models	User	Convert mock-only LLM benchmark into real model comparison
Manual	Run Snort runtime and PCAP tests	User	Convert SKIPPED runtime rows into real PASS/FAIL evidence

3. Codex Tasks - Implementable Now

These tasks do not require Snort or real LLM models to be installed. Codex should implement them directly and run all possible tests.

3.1 Regenerate and verify all stale artifacts

Problem: earlier generated CSVs may still contain old issues, even if the source code was fixed. Codex must regenerate and inspect the artifacts, not only update code.

- Run generation examples again and inspect DNS tunneling, directory traversal, and ambiguous query rows.
- Ensure source_doc_ids are deduplicated while preserving order.
- Ensure explanation classtype is derived from the parsed generated rule, not from a hardcoded family map.
- Ensure benign queries return NO_RULE_RECOMMENDED and are counted as valid benign outputs.
- Re-run clustering, retrieval benchmark, Devoir 3 architecture evaluation, and mock LLM benchmark after fixes.

3.2 Fix source_doc_ids duplication everywhere

The generated examples previously showed duplicate retrieved IDs in some rows. Codex must ensure every returned list of source IDs is unique and stable.

- llm_generator.py: used_source_doc_ids must be deduplicated and must be a subset of retrieved_ids.
- generator.py / architectures.py: source_doc_ids must be deduplicated in all RAG paths, not only LLM path.
- generate_generation_examples.py: CSV output must serialize the cleaned list.
- tests: add or update tests that fail if duplicate source IDs appear.

3.3 Fix explanation consistency

Some old explanations could mention a classtype different from the actual rule. Codex must parse the generated rule and use the real classtype in the explanation.

- Parse classtype from generated_rule when building explanation.
- If parsed classtype is unavailable, say "classtype unavailable" instead of inventing one.
- Add a test using DNS tunneling to confirm explanation does not contradict the rule.

```
PYTHONPATH=src python scripts/generate_generation_examples.py
PYTHONPATH=src python scripts/benchmark_retrieval.py
PYTHONPATH=src python scripts/plot_retrieval_benchmark.py
PYTHONPATH=src python -m snort_rag.run_devoir3
PYTHONPATH=src python scripts/run_clustering_analysis.py
PYTHONPATH=src python scripts/benchmark_llms.py --models mock
PYTHONPATH=src pytest tests/
PYTHONPATH=src python scripts/validate_dataset.py
```

3.4 Update notebook wording

The notebook still contains older wording saying APIs OpenAI/Mistral/Ollama are forbidden. This should be corrected without encouraging black-box generation.

Replace with: Direct black-box LLM generation is not used. The project allows LLM execution only inside a controlled RAG pipeline, where the user query is combined with retrieved documents, a strict schema, local validation, and explicit fallback. Local/offline models such as Ollama are preferred for benchmarking when available.

3.5 Strengthen runtime scripts without claiming success

Codex should improve the Snort scripts so they are ready for real execution and provide better evidence when Snort is missing.

Script	Enhancement
scripts/run_snort_validation.py	Write command attempted, snort_bin_found, config_used, rules_path, timestamp, status, runtime_valid, local_valid, stderr excerpt.
scripts/run_pcap_tests.py	Write pcap path, expected attack, status, alert_count, triggered_sids if available, benign_alert flag, command, stderr excerpt.
scripts/generate_lab_pcaps.py	Document clearly that PCAPs are synthetic lab files, not real captured traffic.
docs/report_section_pcap_runtime_validation.md	Show separate sections for SKIPPED vs executed PASS/FAIL results.

Critical: If Snort is missing, keep SKIPPED. Do not fabricate PASS rows or alert counts.

3.6 Prepare real LLM benchmark execution

Codex cannot guarantee local LLM availability, but it can make the commands, docs, error handling, and output schema ready.

- Ensure benchmark_llms.py gracefully handles Ollama connection errors and writes model rows with status=error instead of crashing the full run.
- Add clear README commands for installing/running Ollama models.
- Ensure llm_benchmark_summary.csv groups by model_spec and prompt_variant.
- Add a report sentence that mock is only a smoke-test client, not final LLM evidence.
- If possible, add prompt variants strict_json and strict_json_repair for comparison.

```
# Manual command that should work after the user installs Ollama:
ollama pull mistral
ollama pull llama3
ollama pull qwen2.5

PYTHONPATH=src python scripts/benchmark_llms.py --models ollama:mistral ollama:llama3 ollama:qwen2.5
--prompt-variants strict_json --out results/llm_benchmark.csv --summary-out
results/llm_benchmark_summary.csv
```

3.7 Improve empirical false-positive reporting

Heuristic false-positive analysis already exists, but empirical benign PCAP results are still missing because Snort execution was skipped. Codex should make the reporting ready for the user run.

Item	Required output
results/false_positive_analysis.csv	Heuristic FP score and suggestions from generated rules.
results/pcap_test_results.csv	Empirical alert counts after Snort replay.
docs/report_section_pcap_runtime_validation.md	Clear table: benign PCAP alert count, malicious PCAP alert count, unexpected alerts, missed detections.
Dashboard	If empirical PCAP results exist, show them; otherwise show "not executed".

3.8 Add final repository health script

Add a script such as scripts/final_health_check.py to summarize whether the repository is final-evidence-ready. It should not require Snort/Ollama, but should detect whether outputs are mock-only or skipped.

```
Expected checks:
- README has no merge conflict markers
- tests pass
- dataset validation passes
- generated_rule_examples.csv has no duplicate source IDs
- llm_benchmark_summary.csv exists
- warn if only model_spec == mock
- snort_runtime_validation.csv exists
- warn if all statuses are SKIPPED
- pcap_test_results.csv exists
- warn if all statuses are SKIPPED
- clustering_metrics.csv exists
- retrieval_backend_proof.json exists
```

3.9 Optional: make submission package script

Create scripts/make_submission_package.py to package only safe project files. It should exclude .env, virtual environments, caches, and unintended raw logs. Lab PCAPs can be included only if they are explicitly synthetic and intended for submission.

4. Manual Tasks After Codex Finishes

These tasks require tools or runtimes that may not exist in Codex. The user must run them locally after Codex finishes the code-side cleanup.

4.1 Install and run real LLMs

Goal: replace mock-only LLM evidence with real local model comparison.

```
# Example Linux/WSL flow after installing Ollama:
ollama serve
ollama pull mistral
ollama pull llama3
ollama pull qwen2.5

cd /mnt/e/projects/snort_rag_rule_generator
PYTHONPATH=src python scripts/benchmark_llms.py --models ollama:mistral ollama:llama3 ollama:qwen2.5

# Then inspect:
cat results/llm_benchmark_summary.csv
```

Success evidence	What to verify
results/llm_benchmark.csv	Rows include ollama:mistral, ollama:llama3, ollama:qwen2.5, not only mock.
results/llm_benchmark_summary.csv	valid_rule_rate, attack_type_accuracy, grounding rate, latency are filled.
Report	State exact models used and limitations honestly.

4.2 Install/run Snort runtime validation

Goal: convert the runtime CSV from SKIPPED to actual PASS/FAIL.

```
# After Snort is installed and snort is in PATH:
cd /mnt/e/projects/snort_rag_rule_generator
PYTHONPATH=src python scripts/run_snort_validation.py --snort-bin snort --rules
data/processed/person1_rules.rules --out results/snort_runtime_validation.csv

# If your Snort requires config:
PYTHONPATH=src python scripts/run_snort_validation.py --snort-bin snort --config "$SNORT_CONFIG" --rules
data/processed/person1_rules.rules --out results/snort_runtime_validation.csv
```

Interpretation: PASS means Snort accepted the rules in that environment. FAIL means rule syntax/config issues must be corrected. SKIPPED means Snort was still unavailable and cannot be claimed as runtime validation.

4.3 Run PCAP replay and empirical benign false-positive test

```
PYTHONPATH=src python scripts/generate_lab_pcaps.py --out tests/pcaps/generated

PYTHONPATH=src python scripts/run_pcap_tests.py --snort-bin snort --rules
data/processed/person1_rules.rules --pcap-dir tests/pcaps/generated --out results/pcap_test_results.csv

# Inspect:
cat results/pcap_test_results.csv
```

Expected evidence	Meaning
Malicious PCAP alerts	Shows detection behavior for attack scenarios.
benign_traffic.pcap alert count	Empirical false-positive signal. Ideally zero or documented.
Triggered SIDs	Links behavior to generated/exported rules.
Snort logs	Auditable runtime output.

5. Final Acceptance Criteria

Codex should consider the repository ready for the user only when all code-side criteria are satisfied. The user should consider it final only after the manual runtime criteria are satisfied.

Category	Ready after Codex?	Final after user?
README and documentation	No conflict markers, no contradictory LLM wording	Final report updated with real model/Snort results
Dataset	Validation script passes, dataset card/manifest present	No extra action unless professor asks
Retrieval	Benchmarks regenerated; backend proof exists	No extra action if environment supports backends
LLM	Mock benchmark passes; real commands documented	Real Ollama/local model rows exist
Generation artifacts	No duplicate source IDs; explanations consistent	Regenerate once after real LLM if needed
Clustering	Metrics and plot exist	No extra action unless report needs polish
Snort runtime	Script ready; SKIPPED if missing	PASS/FAIL rows from real Snort
PCAP empirical FP	PCAP script ready; SKIPPED if missing Snort	Attack/benign alert counts from real Snort
Tests	pytest and dataset validation pass	Run again after manual artifact generation

6. Exact Prompt to Give Codex

Use the following as the implementation prompt for Codex.

You are continuing the `snort_rag_rule_generator` project.

Goal: finish the remaining code-side fixes and evidence preparation after the P0/P1 implementation. Do not fake runtime evidence. If Snort or real LLMs are unavailable, keep outputs clearly marked as SKIPPED or mock-only.

Tasks:

1. Regenerate all result artifacts after the latest code changes.
2. Ensure `generated_rule_examples.csv` has no duplicate `source_doc_ids`.
3. Ensure explanations use the actual parsed rule classtype, especially DNS tunneling.
4. Update notebook/report wording: LLM is allowed only inside controlled RAG; direct black-box LLM use is forbidden.
5. Improve `run_snort_validation.py` and `run_pcap_tests.py` metadata: command, timestamp, binary found, config, PASS/FAIL/SKIPPED, stderr excerpt, triggered SIDs where possible.
6. Improve `benchmark_llms.py` error handling for Ollama models; do not crash the full benchmark if one model is unavailable.
7. Add a `final_health_check.py` script that warns if LLM benchmark is mock-only or Snort/PCAP results are all SKIPPED.
8. Update `docs/report_section_llm_benchmark.md` and `docs/report_section_pcap_runtime_validation.md` with honest final wording and manual commands.
9. Re-run: dataset validation, retrieval benchmark, generation examples, clustering, mock LLM benchmark, tests.
10. Summarize what is fully fixed and what still requires the user to run locally with Ollama/Snort.

Manual-only after Codex:

- Run Ollama models: `ollama:mistral`, `ollama:llama3`, `ollama:qwen2.5`.
- Run Snort runtime validation and PCAP replay in a Snort-capable environment.

Do not claim 99/100 or runtime success until real LLM and Snort evidence exists.

7. Final Command Checklist

Codex should run the first block. The user should run the second block after installing local runtimes.

7.1 Codex-safe commands

```
PYTHONPATH=src python scripts/validate_dataset.py
PYTHONPATH=src python scripts/benchmark_retrieval.py
PYTHONPATH=src python scripts/plot_retrieval_benchmark.py
PYTHONPATH=src python -m snort_rag.run_devoir3
PYTHONPATH=src python scripts/run_clustering_analysis.py
PYTHONPATH=src python scripts/benchmark_llms.py --models mock
PYTHONPATH=src python scripts/generate_generation_examples.py
PYTHONPATH=src python scripts/generate_lab_pcaps.py --out tests/pcaps/generated
PYTHONPATH=src python scripts/run_snort_validation.py --out results/snort_runtime_validation.csv || true
PYTHONPATH=src python scripts/run_pcap_tests.py --out results/pcap_test_results.csv || true
PYTHONPATH=src pytest tests/
PYTHONPATH=src python scripts/final_health_check.py || true
```

7.2 User runtime commands

```
# Real LLM evidence
ollama pull mistral
ollama pull llama3
ollama pull qwen2.5
PYTHONPATH=src python scripts/benchmark_llms.py --models ollama:mistral ollama:llama3 ollama:qwen2.5

# Real Snort evidence
PYTHONPATH=src python scripts/run_snort_validation.py --snort-bin snort --rules
data/processed/person1_rules.rules --out results/snort_runtime_validation.csv

PYTHONPATH=src python scripts/run_pcap_tests.py --snort-bin snort --rules
data/processed/person1_rules.rules --pcap-dir tests/pcaps/generated --out results/pcap_test_results.csv

# Final verification
PYTHONPATH=src pytest tests/
PYTHONPATH=src python scripts/validate_dataset.py
PYTHONPATH=src python scripts/final_health_check.py
```

8. Reporting Rules

- If only mock was executed, write: "The LLM pipeline was smoke-tested with a mock client; real local LLM benchmarking remains to be executed."
- If Snort rows are SKIPPED, write: "Runtime Snort validation was not executed in this environment; only local structural validation was executed."
- If PCAP rows are SKIPPED, write: "Synthetic lab PCAPs were generated, but replay through Snort was not executed because Snort was unavailable."
- If real Snort runs PASS, include exact command, date, Snort version if available, rule count, PASS/FAIL count, warning count, and artifact paths.
- If benign PCAP triggers alerts, do not hide it. Mark it as empirical false-positive evidence and discuss rule refinement.

Final principle: The project should earn points because it is transparent, reproducible, and honest - not because it hides missing runtime conditions.